Architecting the Post-Transformer Paradigm

A Comprehensive Synthesis of Mamba-3, Mixture of Experts,

Recursive Language Models, and Dense Verification

Peter A. Aldrich Jr.

Comprehensive Research Manuscript

March 2026

Keywords: *State Space Models, Mamba-3, Mixture of Experts, Recursive Language Models, Reinforcement Learning, Autonomous Agents, Dense Verification, Hybrid Architectures, Inference Optimization, Agentic AI*

Abstract

Pure Transformer architectures face fundamental scaling limitations: quadratic attention cost in sequence length, uniform parameter activation regardless of task complexity, and no built-in mechanism for iterative self-improvement. Between 2024 and early 2026, a new generation of hybrid models has emerged that combines Mamba's linear-time state-space sequence processing with Mixture of Experts' sparse parameter efficiency, reinforcement-learning-trained reasoning capabilities, and recursive self-correction loops. This paper provides a comprehensive, unified analysis of this multi-paradigm hybrid architecture, synthesizing Mamba-3 (a Multi-Input Multi-Output State Space Model), Latent Mixture of Experts (LatentMoE), Recursive Language Models (RLMs), and a Dense Verification Layer.

We examine the theoretical foundations of these components—including Mamba-3's exponential-trapezoidal discretization, complex-valued state tracking, and MIMO formulation; the parameter efficiency of LatentMoE routing in compressed geometric spaces; and the infinite-context search capabilities of RLMs leveraging external Read-Eval-Print Loop (REPL) environments. Furthermore, we detail critical optimization strategies such as Phased Specialization training, Multi-Token Prediction, and Reinforcement Learning with Dense Utility Rewards (LongR). By evaluating the structural synthesis, scaling algorithms, and deployment lifecycle across disaggregated rack-scale hardware—including production systems such as Tencent's Hunyuan-TurboS (560B parameters), NVIDIA's Nemotron-H (56B), and AI21's Jamba (52B)—this research demonstrates how the proposed multi-paradigm engine fundamentally redefines the performance-efficiency Pareto frontier for autonomous, long-horizon artificial intelligence. We find that the four paradigms address complementary axes of inefficiency and that their benefits compound when composed, with the most integrated systems achieving top-tier benchmark performance at 45% lower cost per token than pure Transformer baselines.

1. Introduction: The Inference-First Epoch

The trajectory of artificial intelligence research has historically been dominated by the scaling laws of dense Transformer architectures. Through self-attention mechanisms, these models established unprecedented capabilities in zero-shot reasoning, semantic generalization, and in-context learning. However, as the deployment of foundation models transitions from simple, single-turn chatbot interactions to complex, unbounded agentic workflows, the fundamental architectural limitations of the standard Transformer have become increasingly intractable.

The core bottleneck lies in the quadratic computational complexity of self-attention and the linear memory scaling of the key-value (KV) cache with respect to sequence length. For a 128K-token context, this means roughly 16 billion pairwise interactions per layer—a cost that grows quadratically as context windows expand toward the million-token frontier demanded by modern applications. The KV cache compounds this problem: storing attention states for long sequences

consumes tens of gigabytes of GPU memory, limiting batch sizes and throughput. In deployment scenarios that mandate massive context processing—such as repository-scale software engineering, long-horizon multi-agent coordination, and recursive self-improvement—the KV cache severely throttles memory bandwidth during autoregressive decoding, leaving expensive computational resources idle.

Beyond raw cost, dense Transformers waste computation by activating all parameters for every token regardless of difficulty. A simple factual lookup receives the same computational budget as a complex multi-step reasoning problem. This uniform allocation is fundamentally inefficient—most tokens in natural language carry low information density and do not require the full model's capacity. Furthermore, dense Transformers invariably suffer from "context rot," a phenomenon where the model's signal-to-noise ratio and recall accuracy degrade precipitously as the context window expands, ultimately paralyzing the network's ability to reason reliably over extended horizons.

These limitations have driven four parallel lines of innovation. State Space Models (SSMs) replace quadratic attention with linear-time recurrences. Mixture of Experts (MoE) decouples total model capacity from per-token compute through sparse activation. Reasoning methods enable models to allocate variable compute at inference time through extended chains of thought. Recursive self-refinement allows models to iteratively improve outputs through feedback loops. To circumvent these profound algorithmic and hardware constraints, the research frontier between 2024 and 2026 rapidly pivoted toward heterogeneous hybrid architectures that synergize the strengths of structurally diverse computational paradigms to decouple state tracking from attention, route computation dynamically, and externalize context into programmatic environments.

The operational requirements and integration protocols for such an architecture are explicitly codified in the Agent Graph Language (AGL) v1.0 specification, which serves as the foundational blueprint for this analysis. The AGL specification defines a recursive, self-verifying prompt engineering agent engineered to decompose complex inputs into structured scaffolds, mandating a four-pillar structural identity: a Mamba-3 selective state space backbone, a Recursive Language Model (RLM) engine, a Mixture of Experts (MoE) domain router, and a Dense Verification layer. This report provides a comprehensive, unified analysis of this Mamba3-MoE-RLM-Dense hybrid architecture, demonstrating how this composite paradigm redefines the performance-efficiency Pareto frontier for unbounded artificial intelligence.

2. State Space Models and the Mamba-3 Backbone

2.1 From S4 to Selective State Spaces

State Space Models formalize sequence processing through continuous-time linear dynamical systems: $x'(t) = Ax(t) + Bu(t)$, $y(t) = Cx(t)$, where A, B, C are learnable matrices and $x(t)$ is a latent state. The breakthrough S4 model (Gu, Goel, and Ré, 2022) showed that initializing A with HiPPO

matrices—derived from optimal polynomial projections of input history—enabled SSMs to solve tasks requiring reasoning over sequences of 16,000+ tokens, which no prior model could handle. S4 operates in dual modes: an O(n) recurrence for inference and an O(n log n) convolution for training, making it both theoretically elegant and practically efficient. However, S4 and its variants used time-invariant parameters—the same A, B, C regardless of input content—which prevented content-based selection and caused SSMs to underperform Transformers on information-dense tasks like language modeling.

Mamba-1 (Gu and Dao, 2023) introduced the selective state space mechanism (S6), making B, C, and the discretization step Δ all input-dependent. This allows the model to selectively propagate or forget information based on token content, with Δ acting as a learned gate controlling how strongly each token overwrites the hidden state. At 2.8B parameters, Mamba matched Transformers twice its size on language modeling while delivering 5× higher inference throughput and linear scaling in sequence length. Subsequently, Mamba-2 (Dao and Gu, 2024) established the State Space Duality (SSD) framework, proving that SSMs and attention are mathematically connected through semiseparable matrices, thereby enabling chunked algorithms that achieved 2–8× speedups over Mamba-1.

2.2 Mamba-3: The Inference-First Paradigm

At the foundational layer of the hybrid stack lies Mamba-3, a state space model engineered fundamentally around an "inference-first" philosophy. Rather than optimizing training throughput, Mamba-3 addresses the "cold GPU" problem: during autoregressive decoding, modern hardware sits idle because the arithmetic intensity of standard SSM decoding falls far below the compute-bound regime of modern GPUs. Despite the advances of Mamba-1 and Mamba-2, previous iterations heavily optimized for pre-training speed by simplifying the underlying recurrence mechanisms. Consequently, earlier models exhibited severe deficiencies in formal algorithmic logic, state-tracking, and tasks requiring high-fidelity memory retention, often performing no better than random guessing on arithmetic and parity-checking benchmarks. Mamba-3 rectifies these structural limitations through a triad of core methodological innovations derived from classical control theory.

2.3 Exponential-Trapezoidal Discretization

Because SSMs are continuous-time systems, they must be mathematically discretized to process discrete digital sequences. Previous iterations utilized a first-order heuristic known as exponential-Euler discretization, which carries a truncation error proportional to the square of the time step (Δ^2). Mamba-3 replaces this heuristic with an exponential-trapezoidal discretization scheme. This advanced approach provides a second-order accurate approximation of the state-input integral, significantly reducing the truncation error to the cube of the time step (Δ^3). When mathematically expanded, this discretization transforms the discrete recurrence from a standard

two-term update into a more complex three-term update equation incorporating the current input, the previous state, and an implicit, data-dependent, width-2 convolution applied directly to the input sequence. By coupling this implicit convolution with explicitly initialized bias terms for the structured matrices, the Mamba-3 architecture entirely circumvents the need for the short causal convolutions (Conv1D layers) that were previously deemed mandatory for recurrent language models, streamlining the hardware implementation and creating a substantially smoother data flow through the GPU memory hierarchy.

2.4 Complex-Valued State Space and Rotational Dynamics

A critical failure mode of earlier real-valued SSMs was their inability to maintain high-fidelity representations of positional and sequential states. To resolve the capability degradation observed in formal state-tracking tasks, Mamba-3 transitions the hidden state matrix from the real mathematical domain to the complex domain. Within a complex-valued state space, the recurrence update rule functions mathematically as a continuous rotation in the latent space. This "RoPE trick" establishes a direct functional and geometric equivalence to data-dependent Rotary Positional Embeddings (RoPE), a mechanism that is standard in highly capable dense Transformers. This structural evolution empowers the recurrent network to maintain high-fidelity representations of sequence states and execute synthetic state-tracking operations with near-perfect accuracy, effectively eliminating the catastrophic forgetting that plagues real-valued scalar transitions and bridging the historical "logic gap" of linear models. Notably, Mamba-3 can solve parity and modular arithmetic tasks where Mamba-2 performs at random chance.

2.5 Multi-Input Multi-Output (MIMO) Formulation

The most consequential hardware optimization introduced by Mamba-3 addresses the "cold GPU" problem inherent in autoregressive decoding. Standard Single-Input Single-Output (SISO) SSMs are severely memory-bound during the decoding phase because the recurrent state update relies on an outer-product operation. This creates a massive arithmetic bottleneck where the immense computational power of GPU tensor cores remains largely idle while the system waits for comparatively slow VRAM data transfers. Mamba-3 generalizes the state dynamics to a Multi-Input Multi-Output (MIMO) formulation, which transforms the state update from a memory-bound outer product into a compute-bound matrix-multiplication operation. By expanding the projection matrices to a designated rank, the model executes parallel state updates, drastically increasing the network's arithmetic intensity—defined as the ratio of floating-point operations (FLOPs) to memory traffic. This allows the MIMO architecture to compute up to four times more decoding FLOPs by absorbing the "free" idle compute cycles that would otherwise be wasted.

The empirical results of the MIMO formulation demonstrate a definitive shift in the performance-efficiency frontier. At the 1.5-billion-parameter scale, the Mamba-3 MIMO variant achieves a 57.6% average downstream accuracy across a suite of language modeling benchmarks,

representing a 2.2-percentage-point leap over the industry-standard Transformer baseline and a 1.9-point improvement over Mamba-2. Furthermore, across state-size experiments, Mamba-3 MIMO achieves the perplexity and modeling capability of a Mamba-2 model with twice the internal state size, effectively doubling the inference throughput for the same hardware footprint without incurring any penalty to wall-clock decoding latency. At a sequence length of 16,384 tokens, Mamba-3 clocks prefill and decode operations at 140.61 seconds, compared to a staggering 976.50 seconds for Llama-3.2-1B running on vLLM hardware—representing nearly a 7× speedup over dense Transformer baselines.

3. Massive Parameter Scaling: Latent Mixture of Experts

3.1 Core Mechanism and Routing Strategies

While the Mamba-3 MIMO backbone provides exceptionally efficient local sequence routing and linear-time temporal processing, purely recurrent architectures inherently lack the massive parameter capacity required to encode broad world knowledge and execute complex combinatorial logic. To scale the model's absolute capacity without triggering a proportional, quadratic increase in inference latency, the architecture integrates a Mixture of Experts (MoE) module. In an MoE paradigm, standard dense feed-forward networks (FFNs) are periodically replaced by dynamic routing layers. These layers employ a gating network to assign each individual token to a sparse subset of highly specialized expert sub-networks, fundamentally decoupling the total parameter count from the active per-token computation.

This strategy has been highly successful at scale. Mixtral 8×7B houses 46.7 billion parameters but activates only 12.9 billion per token, achieving performance comparable to Llama 2 70B at a fraction of the inference cost. DeepSeek-V3 activates merely 37 billion out of 671 billion total parameters. MoE routing strategies vary from simple top-K token-choice routing to Expert Choice routing—where experts select their preferred tokens for guaranteed load balance—and auxiliary-loss-free balancing strategies that utilize dynamic per-expert bias terms adjusted based on recent load. Load balancing remains a central challenge; DeepSeek-V3 pioneered auxiliary-loss-free balancing, achieving both better balance and better model quality.

3.2 The LatentMoE Configuration

However, scaling traditional Expert Parallelism (EP) to hundreds of experts across distributed GPU clusters typically creates a devastating all-to-all communication bottleneck. In standard dense MoE models, full-dimensional hidden states must be transmitted across the inter-node network fabric (e.g., NVLink) to reach the GPUs hosting the designated experts, leading to severe synchronization stalls. To optimize this process for massive scale, the hybrid architecture employs a specialized LatentMoE configuration. Rather than routing full-dimensional tokens, the LatentMoE mechanism utilizes a learnable down-projection matrix to compress the input tokens from the standard hidden

dimension into a highly reduced latent space. The entirety of the expert computation and the all-to-all token routing occurs within this compressed geometric space. Once the highly specialized experts process the token, the output is projected back up to the full hidden dimension.

This geometric compression drastically reduces the volumetric payloads and inter-GPU communication latency. By reinvesting these bandwidth and weight-loading savings into computational capacity, the LatentMoE architecture enables Wide-Expert Parallelism, supporting massive scaling across rack-scale domains. This vast reservoir of specialized, conditionally activated parameters provides the foundational knowledge base required to resolve domain-specific logic, formal mathematics, and granular coding syntax.

Table 1: Structural Disparities Between Dense and Latent Routing Paradigms

Routing Architecture	Input Projection	Computation Space	Inter-GPU Payload	Active Experts / Token	Total Parameter Scale
Dense MoE	Identity Matrix	Full Hidden Dimension	Maximal (All-to-All)	2 to 4	~30B to 50B
LatentMoE	Learnable Down-Projection	Compressed Latent Space	Minimal (Reduced by geometric ratio)	Up to 22	120B+

4. Infinite Context Orchestration: Recursive Language Models

4.1 The REPL Paradigm

Despite the linear memory scaling afforded by the Mamba-3 backbone, embedding millions of tokens directly into a neural network's internal hidden state inevitably induces context rot. Information retrieval benchmarks demonstrate that as prompt length increases, internal attention mechanisms and recurrences become saturated. The model's ability to accurately recall information from the context decreases, leading to a loss of critical details and an inability to perform robust multi-hop deductions. Real-world tasks reveal 40–80% performance degradation on massive documents, even when the model theoretically has access to all tokens.

The Recursive Language Model (RLM) paradigm resolves this absolute limit by fundamentally altering the inference-time interaction framework, shifting from static sequence modeling to active environmental orchestration. An RLM transitions the input prompt from an internal tensor sequence to an external environmental variable. The core hybrid neural model is encapsulated within an isolated Read-Eval-Print Loop (REPL) programming sandbox, typically utilizing a persistent Python environment, as operationalized by platforms like Prime Intellect's RLMEnv.

When a user submits an unbounded prompt—potentially exceeding ten million tokens—the text is not fed directly into the model's context window. Instead, it is loaded into the REPL memory as a string variable. The root language model receives only constant-size metadata regarding the environment, such as the variable handle, the total string length, and a brief preview, effectively circumventing the standard context window limitations. The model is then explicitly trained to programmatically examine, decompose, and traverse the external data by generating executable code. The RLM supports symbolic recursion, providing the root model with functions—such as llm_batch—to invoke parallel sub-instances of itself over specifically sliced snippets of the context. This methodology transforms probabilistic, diffuse attention over millions of tokens into deterministic algorithmic search, utilizing traversal strategies such as binary narrowing, explicit regular expression filtering, and chunk-by-chunk summarization.

4.2 Benchmark Superiority and Context Extension

Empirical evaluations validate the supremacy of the RLM approach over standard context scaling. The architecture is frequently subjected to the Object-Oriented Long-Context (OOLONG) benchmark, a rigorous suite that eschews simplistic "needle-in-a-haystack" retrieval in favor of tasks requiring multi-chunk analysis, precise numerical counting, temporal event aggregation, and distributional analysis over highly noisy, unstructured conversational data. Under these stringent conditions, standard frontier models experience severe accuracy drop-offs. On a complex 132K-token OOLONG split, a baseline GPT-5 model achieved a score of 30.2, while an RLM utilizing a much smaller GPT-5-mini model achieved a score of 64.7—a 114% improvement over the massive dense baseline, executed at a comparable or lower overall cost per query. The advantages scale geometrically with context size: on BrowseComp-Plus tasks requiring multi-hop analysis across 1,000 documents (exceeding 5 million tokens), the RLM architecture maintained 100% accuracy while standard models and RAG pipelines degraded significantly.

Table 2: Context Extension Benchmark Superiority (OOLONG & BrowseComp-Plus)

Model Framework	OOLONG Accuracy (131K Tokens)	BrowseComp-Plus Accuracy (~5M Tokens)
GPT-5 Base (Dense)	30.2	~40% (Truncated degradation)
GPT-5 + RAG (BM25)	N/A	~60%
ReAct + GPT-5 + RAG	N/A	~80%
RLM (GPT-5-mini)	64.7 (+114% vs Base)	N/A
RLM (GPT-5 Root)	N/A	100% (No degradation)

4.3 The Limits of Recursion: The Overthinking Anomaly

While inference-time recursion extends functional context limits by orders of magnitude, unbounded recursion introduces severe operational vulnerabilities. Research into recursive depth scaling reveals a compelling and problematic phenomenon defined as "overthinking." While a depth-1 RLM effectively boosts accuracy on complex reasoning tasks, applying deeper recursion (depth-2 or greater) paradoxically degrades performance. Excessive recursion causes the agent to lose sight of the primary objective, exponentially inflating execution time—for example, scaling a simple query from 3.6 seconds to 344.5 seconds—and incurring massive token costs without yielding corresponding accuracy gains. Furthermore, analysis identifies a heightened risk of infinite looping, redundant API calls, and prompt injection vulnerabilities triggered by the autonomous, multi-pass execution of REPL code. This non-linear relationship between recursive depth and output quality mathematically necessitates a rigorous, externalized mechanism for halting conditions and logic verification.

5. Algorithmic Grounding: The Dense Verification Layer

The inherent operational risk of combining sparse, dynamic LatentMoE routing with the autonomous code generation of an RLM is the rapid accumulation of hidden state errors. Over the course of extended recursive loops, minor probabilistic deviations can cascade into severe logical divergence, hallucinated Python variables, or unbreakable execution cycles. Crucially, observations of intrinsic self-correction models indicate that Large Language Models cannot reliably correct their own reasoning without external feedback. When the evaluating agent and the generating agent share identical parameter spaces and blind spots, iterative loops frequently just rearrange errors rather than eliminating them. Formalized as a Markov chain, approximately 75% of reachable improvement is captured in the first two rounds of self-reflection, with rapidly diminishing returns leading to reward hacking and score inflation.

To rigorously counteract this entropy, the architecture embeds a Dense Verification Layer into the execution pipeline. While the generation and exploration phases utilize sparse MoE routing to maximize speed and parameter efficiency, the Dense Verification pass forces a full-capacity, high-attention evaluation over the drafted response and the accumulated REPL state history. This layer acts as an uncompromising system-level supervisor. The verification protocols involve strict factual grounding validation, ensuring that every claim generated by the sparse experts can be explicitly traced back to the established REPL variables, thereby eliminating fabricated logic. Additionally, the layer conducts coherence checks, confirming that all structural dependencies and syntactic formatting rules are mathematically satisfied.

To prevent this dense evaluation pass from creating a severe computational bottleneck, the architecture employs a unified batch scheduling system. Verification tasks are programmatically delayed by a single iteration step, removing the dense computational overhead from the critical inference path of the subsequent recursive generation and ensuring balanced resource utilization across the GPU cluster. In a production environment, tracking the frequency at which the Dense

Verification Layer rejects a drafted response provides a real-time, mathematical indicator of overall system health, allowing automated regression gates to halt operations if the underlying agent begins to drift.

6. Architectural Synthesis: The Mamba3-MoE-RLM Blueprint

6.1 Structural Interleaving and Data Flow

The synthesis of these diverse computational paradigms creates a highly sophisticated, multi-layered agentic engine capable of unbounded reasoning. The underlying neural backbone eschews homogeneity, instead interleaving Mamba-3 MIMO blocks with LatentMoE attention blocks to construct a foundation that is computationally lean yet exceptionally rich in factual capacity. The optimal topological configuration requires strategic placement of the attention mechanisms. Because Mamba-3 processes sequences temporally, it excels at local pattern recognition and linear scaling; however, pure SSMs can still struggle with global information retrieval over extremely long spans. To resolve this, the architecture injects global attention layers at periodic intervals to act as global information anchors. A standard configuration involves a ratio of four to seven Mamba-3 blocks for every one global attention LatentMoE block.

As data flows through the network, the Mamba-3 layers rapidly process the temporal dynamics and local syntax, operating highly efficiently via the MIMO state updates. When the sequence reaches an attention-MoE layer, the tokens are evaluated globally. The MoE router projects the representations into the latent space, distributes them across the massive expert pool, and aggregates the specialized knowledge back into the sequence stream. This neural backbone is continuously orchestrated by the external RLM framework, which dictates the macro-level interaction with the massive user prompts.

6.2 How the Four Components Complement Each Other

Each component addresses a different axis of inefficiency, and their benefits multiply when combined. SSM layers reduce $O(n^2)$ attention cost to $O(n)$, enabling million-token contexts without KV cache explosion. MoE reduces per-token compute by activating only a fraction of parameters. Together, an SSM-MoE hybrid achieves both linear-time sequence processing and sparse parameter activation, fitting massive parameter models on constrained hardware. Reasoning through extended chain-of-thought generates thousands of intermediate tokens; without efficient base architectures, this is prohibitively expensive. MoE makes reasoning economically feasible, and SSMs make long reasoning traces computationally tractable. Even the best reasoning models produce errors; recursive refinement loops combined with Dense Verification provide a mechanism for error correction that single-pass generation inherently lacks.

Table 3: Comparative Performance of the Hybrid Architecture Against Conventional Paradigms

Architectural Metric	Pure Dense Transformer	Pure Mamba-2 SSM	Hybrid Mamba3-MoE-RLM
Computational Complexity	$O(N^2)$	O(N)	O(N) local + algorithmic search via RLM
Memory Bottleneck (KV Cache)	Severe linear growth	Negligible (Constant State)	Minimal (Periodic anchor caching)
Parameter Efficiency	Low (100% active per token)	Low (Hard capacity limits)	Very High (Sparse LatentMoE)
Effective Context Limit	~128K to 2M tokens	~1M tokens	Unbounded (10M+ via REPL)
Hardware Utilization	Poor during decode	Low arithmetic intensity	Exceptional (MIMO leverages idle tensor cores)
Algorithmic State Tracking	Excellent	Poor	Excellent (Complex-valued transitions)

7. Training the Hybrid Architecture: Optimization and Reinforcement

7.1 Phased Specialization

Training a highly heterogeneous architecture comprising continuous temporal recurrence (Mamba-3), sparse distributed routing (LatentMoE), and periodic dense global attention necessitates profoundly specialized optimization strategies. Conventional joint optimization—where all network parameters are updated simultaneously using a uniform learning rate—frequently results in catastrophic training failure for hybrid models. The primary cause is gradient interference and learning-speed asymmetry. Softmax attention layers possess inherently stable optimization surfaces and shorter effective gradient paths, allowing them to learn rapidly and dominate early gradient updates. Conversely, Mamba-3 SSM modules and MoE routers require significantly more iterations to stabilize their complex-valued transition matrices and routing distributions. Under a standard joint training regime, the faster attention modules resolve local features so quickly that the SSM module is effectively ignored by the optimizer, treated as noise, and rendered redundant, leading to severe perplexity degradation and extreme variance across training seeds.

To resolve this optimization conflict, the architecture relies on Phased Specialization, an optimization-aware training strategy utilizing differential learning rates to temporally decouple module updates. Phased Specialization functions as an approximate block coordinate descent algorithm. The training schedule is bifurcated into distinct temporal phases. During Phase 1

(Attention Lead, epochs 1–4), attention and MoE layers receive a standard high learning rate (3×10^{-4}) while Mamba-3 layers receive a minimal rate (1×10^{-5}), allowing the attention mechanism to establish robust feature extraction pipelines. Once these representations stabilize, the system shifts to Phase 2 (SSM Specialization, epochs 5–8), reducing the attention learning rate and maximizing the Mamba-3 learning rate. Because the input statistics flowing from the attention layers are now nearly fixed, the SSM can adapt to process these structured representations without competing for the gradient signal.

A crucial implementation principle is the strict avoidance of a "hard freeze" (setting any learning rate to absolute zero). A persistent minimal update acts as a compatibility signal, ensuring disparate modules continuously align and adapt to minor shifts in output statistics. Empirical validation demonstrates that Phased Specialization reduces Conflict Rates by 53%, reduces cross-seed training variance by 70%, and yields a massive 34-point perplexity improvement over standard joint training, establishing it as a mandatory protocol for stabilizing hybrid loss surfaces.

7.2 Architecture Distillation and Speculative Decoding

Beyond from-scratch training, the efficiency of hybrid models is frequently bootstrapped via cross-architecture distillation. Frameworks such as Llamba demonstrate the viability of distilling knowledge from massive dense Transformers (e.g., Llama-3) directly into Mamba architectures. Utilizing algorithms like MOHAWK, these models achieve comparable benchmark performance using less than 0.1% of the training data typically required for models of similar size, vastly accelerating the deployment pipeline.

To further optimize inference speeds, the architecture incorporates Multi-Token Prediction (MTP) heads. Traditional pre-training optimizes the model to predict solely the next adjacent token. MTP extends this objective by adding specialized, weight-shared layers that force the model to predict several tokens into the future simultaneously. This auxiliary training objective forces the Mamba-3 backbone to learn deeper semantic representations during pre-training and seamlessly enables native speculative decoding during deployment. Because the model has been explicitly trained to generate multi-token trajectories, it can draft future sequences instantly, which are then rapidly verified by the Dense Verification Layer, leading to profound accelerations in autoregressive throughput.

7.3 Reinforcement Learning with Dense Utility Rewards (LongR)

Endowing the foundational hybrid architecture with the capacity to execute recursive, multi-step code generation requires sophisticated post-training via Reinforcement Learning. While prompting methodologies like Chain-of-Thought provide baseline reasoning, genuine deductive capability emerges from RL training methodologies. Models such as DeepSeek-R1 have demonstrated that employing Group Relative Policy Optimization (GRPO)—a variant that eliminates the critic model by estimating advantages from group-relative rewards—can produce extended internal reasoning and

emergent metacognitive behaviors, including self-verification, reflection, backtracking, and spontaneous "aha moments." The integration of Process Reward Models (PRMs) provides step-level feedback, significantly outperforming outcome-based supervision.

The fundamental challenge in applying RL to long-context RLM frameworks is the extreme sparsity of the reward signal. If an agent spends twenty iterations autonomously exploring a massive codebase and receives a binary outcome reward only at the final step, the gradient signal vanishes before it can optimize the early, critical contextual grounding decisions. To solve this vanishing gradient problem, the training pipeline implements the LongR (Dense Utility Rewards) framework. LongR augments the traditional outcome-based reward function with a continuous, dense verification signal applied to every intermediate step of the RLM's trajectory. The system implements a dynamic "Think-and-Read" mechanism, explicitly training the model to interleave internal reasoning steps with active REPL environmental extraction. The reward function quantifies the information-theoretic utility of every retrieval action using a Relative Information Gain metric, calculated using token-level Negative Log-Likelihood (NLL) derived from an external verification model.

Crucially, the LongR framework carefully calibrates the density of this reward. Empirical testing demonstrates that using an "Absolute Density" formulation—where raw information gain is unbounded—causes the policy to hack the reward, generating unnecessarily verbose code to artificially inflate prior uncertainty. Conversely, a "Threshold Density" formulation lacks the necessary granularity. By utilizing relative, constrained utility rewards, combined with a curriculum learning strategy that progressively scales context from 16K to 32K tokens, the framework heavily penalizes superficial text extraction and forces the Mamba-3 backbone to learn highly efficient, deductive search patterns. This rigorous RL pipeline directly yields a 9% performance gain on the LongBench v2 evaluation and establishes consistent robustness against distractor inputs.

8. Scaling, Hardware Parallelism, and Deployment

8.1 Expert Parallelism and Latent Routing

Scaling a hybrid architecture possessing over 120 billion total parameters across distributed clusters introduces severe infrastructure challenges. The cornerstone of the scaling strategy is Expert Parallelism (EP). In standard data parallelism, the entire model is replicated across every GPU. EP distributes individual experts across distinct GPUs; when tokens reach an MoE layer, the router determines optimal experts and dispatches tokens across the network fabric to the specific GPUs hosting those experts. Because the LatentMoE design down-projects tokens into a compressed latent dimension prior to routing, the volumetric payload transmitted across the inter-GPU fabric is drastically reduced, enabling Wide-Expert Parallelism (Wide-EP) across rack-scale domains. Parallelizing the continuous SSM layers requires selective parameter partitioning, isolating

recurrent updates to individual GPUs and utilizing quantized AllReduce operations. Sequence Parallelism is layered on top to distribute unbounded RLM generation tokens across devices.

8.2 Disaggregated Serving

Moving these massive hybrid architectures into production requires abandoning traditional serving frameworks, which invariably colocate the compute-bound prefill phase with the memory-bound autoregressive decode phase, leading to severe resource fragmentation. The hybrid architecture necessitates Disaggregated Serving via specialized inference orchestration, completely decoupling prefill operations from decode operations and assigning them to distinct, optimized GPU nodes. The prefill nodes execute massive initial prompt comprehension, generating initial hidden states streamed to dedicated decode nodes. Because the Mamba-3 MIMO variant inherently possesses high arithmetic intensity, the decode nodes can utilize GPU tensor cores at near-maximum capacity. This disaggregated flow requires immense inter-node bandwidth, making the architecture highly suited for deployment on rack-scale designs such as the NVIDIA GB200 NVL72.

8.3 Kernel-Level Execution Paradigms

To extract maximum performance, the hybrid operations bypass standard PyTorch abstractions in favor of bespoke, hardware-aware kernel implementations leveraging a polyglot kernel stack. Triton kernels execute the complex-valued SISO state transitions with seamless fusion of rotary positional embeddings. TileLang optimizes the granular memory hierarchies for MIMO prefill matrix multiplications. CuTe DSL fuses MoE gating directly with MIMO state projections to eliminate redundant VRAM read/write cycles on the critical decode path.

9. Hardware Resource Analysis: The Mamba3-MoE-RLM-Dense Architecture

The preceding sections have described each component of the proposed Mamba3-MoE-RLM-Dense Verification hybrid in theoretical terms. However, a rigorous deployment decision demands concrete, per-parameter hardware accounting—not for existing production models that approximate parts of the design, but for the fully integrated four-pillar architecture itself. This section defines a reference configuration of the complete Mamba3-MoE-RLM-Dense system, decomposes its resource consumption layer by layer and component by component, and compares the resulting hardware envelope against equivalently capable dense Transformer baselines at multiple parameter scales.

9.1 Reference Configuration of the Mamba3-MoE-RLM-Dense Architecture

To make the hardware analysis concrete, we define a reference instantiation of the full four-pillar architecture at two scales—a Research-Scale variant (approximately 60B total parameters, 8B active

per token) and a Production-Scale variant (approximately 550B total parameters, 55B active per token). Both configurations share the same structural blueprint; only the dimensional hyperparameters change. The architecture is not a monolithic neural network serving queries in a single forward pass. Rather, it is a multi-process system comprising four distinct resource consumers that operate in coordination: the Mamba-3 MIMO backbone, the LatentMoE routing layer, the external REPL sandbox hosting the RLM execution engine, and the Dense Verification supervisor. Each of these components draws from different hardware pools—GPU compute, GPU memory, CPU memory, and inter-node bandwidth—and their combined envelope differs fundamentally from a standard Transformer serving stack.

Table 6: Reference Mamba3-MoE-RLM-Dense Architecture Specification

Architectural Component	Research-Scale (60B / 8B active)	Production-Scale (550B / 55B active)
Mamba-3 MIMO Layers	56 layers; hidden dim 4,096; state dim N=64; complex-valued; MIMO rank R=4	96 layers; hidden dim 8,192; state dim N=128; complex-valued; MIMO rank R=8
Global Attention Anchor Layers	8 layers (1:7 ratio); grouped-query attention; 32 heads / 8 KV heads	12 layers (1:8 ratio); grouped-query attention; 64 heads / 16 KV heads
LatentMoE FFN Layers	32 MoE layers; 16 experts + 1 shared per layer; top-2 routing; latent dim 512 (8× compression from hidden 4,096)	48 MoE layers; 64 experts + 2 shared per layer; top-2 routing; latent dim 1,024 (8× compression from hidden 8,192)
Total Neural Parameters	~60B total (Mamba-3: ~14B; Attention: ~4B; MoE experts: ~38B; shared: ~4B)	~550B total (Mamba-3: ~52B; Attention: ~18B; MoE experts: ~440B; shared: ~40B)
Active Parameters Per Token	~8B (Mamba-3: 14B + Attn: 4B + 2 active experts: ~4.8B + shared: ~4B = ~27B per full layer pass; amortized per-token: ~8B)	~55B (Mamba-3: 52B + Attn: 18B + 2 active experts: ~13.8B + shared: ~40B; amortized per-token: ~55B)
RLM REPL Sandbox	Persistent Python 3.12 environment; 16 GB CPU RAM allocation; llm_batch parallelism: 4 sub-instances	Persistent Python 3.12 environment; 64 GB CPU RAM allocation; llm_batch parallelism: 16 sub-instances
RLM Root Context Window	8K tokens (metadata only: variable handles, string lengths, REPL state)	16K tokens (metadata only: expanded state tracking for deeper recursion)
RLM Effective Context Reach	10M+ tokens (via programmatic REPL traversal; actual prompt stored in CPU RAM as string variable)	100M+ tokens (via parallel sub-instance traversal across distributed REPL pools)
Dense Verification Layer	Full-capacity forward pass using all 8B active params; triggered once per RLM iteration; batch-delayed by 1 step	Full-capacity forward pass using all 55B active params; triggered once per iteration; pipelined on dedicated GPU partition
Recursive Depth Budget	2–3 refinement iterations (based on Yang et al. convergence analysis); hard-stop at depth 5	2–3 refinement iterations; hard-stop at depth 5; adaptive early exit via verification confidence threshold

This specification reveals that the Mamba3-MoE-RLM-Dense architecture is not a single neural network with a unified resource profile. It is a heterogeneous multi-process system where the neural backbone (Mamba-3 + Attention + LatentMoE) consumes GPU compute and HBM memory, the RLM REPL sandbox consumes CPU RAM and generates recursive token traffic, and the Dense

Verification Layer consumes additional GPU cycles on a delayed schedule. A dense Transformer, by contrast, is a single-process system where all resource consumption occurs in one forward pass. This structural difference means that per-parameter comparisons must account for the RLM's token multiplication factor and the verification layer's overhead—costs that have no analog in standard Transformer serving.

9.2 GPU Memory Decomposition Per Component

In a dense Transformer, GPU memory consumption during inference is dominated by two elements: model weights and the KV cache. For the Mamba3-MoE-RLM-Dense architecture, memory consumption is distributed across four distinct pools, each with fundamentally different scaling behavior. The first pool is the Mamba-3 MIMO layer weights, which consume memory proportional to their parameter count but generate no KV cache whatsoever. Instead, each Mamba-3 layer maintains a constant-size recurrent state: for a state dimension N=64 with complex-valued entries and MIMO rank R=4, each layer stores approximately $(2 \times N \times d \times R) = 2 \times 64 \times 4{,}096 \times 4 \approx 2.1$ MB in FP16 at the research scale. Across 56 Mamba-3 layers, the total recurrent state is approximately 118 MB—constant regardless of whether the sequence is 4K or 10M tokens long. This is the single most important memory advantage of the architecture: where a dense Transformer's KV cache grows linearly with sequence length, the Mamba-3 backbone's state remains fixed.

The second memory pool is the global attention anchor layers' KV cache. Because the architecture retains 8 attention layers (at the research scale), these layers do generate cache entries that grow linearly with sequence length. However, the cache covers only 8 of the 96 total layers (8.3%), reducing it by approximately 91.7% compared to a fully attentive Transformer of equal depth. Additionally, grouped-query attention (32 query heads sharing 8 KV heads) further compresses each layer's cache by 4×. The net result is that the attention cache in the research-scale Mamba3-MoE-RLM system at 128K tokens consumes approximately 2.1 GB, compared to approximately 42 GB for a comparably sized 80-layer dense Transformer—a 20× reduction.

The third memory pool is the LatentMoE expert weights. The research-scale configuration houses 16 experts per MoE layer across 32 layers, totaling approximately 38B expert parameters. However, because top-2 routing activates only 2 of 16 experts per token, only the routing infrastructure and the two selected experts must reside in high-bandwidth GPU HBM for the current token. Dormant experts can be staged in CPU DRAM or NVMe and swapped via Expert Parallelism. In a single-GPU deployment, the active expert slice consumes approximately 9.6 GB (2 experts × ~4.8B params × 2 bytes FP16), while the full expert pool requires distributed storage across the cluster. The LatentMoE's 8× latent compression means that inter-GPU routing payloads are reduced from the full 4,096-dimensional hidden state to a 512-dimensional latent vector per token, cutting all-to-all communication volume by 87.5%.

The fourth memory pool—unique to this architecture and absent from any pure Transformer system—is the RLM REPL sandbox. This sandbox runs as a separate CPU process (not on the GPU)

and allocates its own memory for storing the externalized context. When a user submits a 5-million-token prompt, this text is held entirely in CPU RAM as a Python string variable, consuming approximately 10 GB at 2 bytes per Unicode character. The neural backbone never sees this full text in its context window; it receives only constant-size metadata handles (variable name, string length, preview snippet) totaling approximately 200–500 tokens. The REPL also maintains a Python interpreter state (local variables, intermediate results, function definitions) that typically consumes 0.5–2 GB of additional CPU memory. This CPU-side allocation is critical: it means the RLM's effective context reach of 10M+ tokens imposes zero additional GPU memory cost beyond the fixed Mamba-3 recurrent state and the small attention cache for the 8K-token root context window.

Table 7: GPU and CPU Memory Decomposition — Mamba3-MoE-RLM-Dense vs. Dense Transformer (Research-Scale, 128K Context)

Memory Pool	Dense Transformer 70B	Dense MoE 670B (37B active)	Mamba3-MoE-RLM 60B (8B active)	Scaling Behavior in Mamba3-MoE-RLM
Model Weights (GPU HBM)	140 GB (all active)	~74 GB (active slice)	~16 GB (active slice)	Scales with active params only; dormant MoE experts offloaded to CPU DRAM
KV Cache (GPU HBM)	~42 GB (80 attn layers)	~2.8 GB (MLA compressed)	~2.1 GB (8 attn layers + GQA)	Grows linearly but only with 8 layers, not 80; 91.7% smaller than dense
Mamba-3 Recurrent State (GPU HBM)	N/A (0 MB)	N/A (0 MB)	~118 MB (constant)	CONSTANT regardless of sequence length; does not grow at 4K, 128K, or 10M tokens
REPL Sandbox (CPU RAM)	N/A (0 MB)	N/A (0 MB)	~10–16 GB (user prompt + interpreter state)	Scales linearly with externalized prompt size; allocated on CPU, not GPU; zero GPU cost
Dense Verification Buffer (GPU HBM)	N/A (0 MB)	N/A (0 MB)	~2 GB (pipelined; shares backbone weights; caches draft + REPL trace)	Fixed per-iteration cost; reuses backbone parameters; adds ~2 GB activation buffer
Peak GPU VRAM (Total)	~200 GB	~320 GB	~20 GB	10× smaller than dense 70B; fits single consumer-grade GPU at FP8/INT4
Peak CPU RAM (Total)	~8 GB (OS + overhead)	~16 GB (expert staging)	~26 GB (16 GB REPL + 8 GB expert staging + 2 GB OS)	Higher CPU demand than dense due to REPL; requires host machine with adequate system RAM
Minimum GPUs (80 GB HBM)	4× H100	8× H100	1× H100 (FP16) or 1× RTX 4090 (INT4)	Single-GPU deployment viable; CPU RAM is the binding constraint, not GPU

The most striking finding in this decomposition is the inversion of the binding resource constraint. For a dense Transformer, the GPU's HBM capacity is almost always the bottleneck: the 70B model's 200 GB total exceeds any single GPU. For the Mamba3-MoE-RLM-Dense architecture, the GPU

requirement drops to approximately 20 GB—well within a single H100—but the CPU RAM requirement rises to 26 GB or more, driven by the REPL sandbox. In practical deployment, this means the Mamba3-MoE-RLM-Dense system requires a host machine with adequate system memory (64–128 GB for production workloads) rather than expensive multi-GPU clusters. The GPU cost per serving node drops by 4–16×, while the CPU memory cost increases by approximately 2–3×—a highly favorable trade-off given that CPU RAM costs approximately $3–5 per GB compared to $50–100 per GB-equivalent of GPU HBM.

9.3 Computational Cost Per Token: The Recursive Multiplier

In a dense Transformer, the computational cost per user-visible output token is straightforward: approximately 2P FLOPs for one forward pass, where P is the parameter count. A 70B dense Transformer therefore costs approximately 140 GFLOPs per output token. The Mamba3-MoE-RLM-Dense architecture introduces a fundamentally different cost structure because a single user-visible output token may require multiple internal forward passes through the neural backbone. This recursive token multiplication is the critical cost dimension that has no analog in standard Transformer serving and must be explicitly modeled to produce accurate hardware projections.

The total compute cost per user-visible token in the Mamba3-MoE-RLM-Dense system decomposes into four additive terms. The first is the base generation pass through the Mamba-3 + LatentMoE backbone. With 8B active parameters (research scale), this costs approximately 16 GFLOPs per generated token—already 8.75× cheaper than the 70B dense baseline. The Mamba-3 layers contribute approximately 2 × 14B = 28 GFLOPs per full-sequence pass, but because the SSM recurrence processes each token in $O(d \times N)$ rather than $O(d^2)$ attention, the per-token amortized cost is substantially lower, approximately 0.5 GFLOPs per token at the research scale. The LatentMoE layers cost approximately 2 × 4.8B (two active experts) = 9.6 GFLOPs per token. The 8 attention layers contribute approximately 2 × 4B = 8 GFLOPs per token but only for 8 of 96 layers, amortized to approximately 0.7 GFLOPs per token.

The second cost term is the RLM's recursive token overhead. When the RLM activates—which occurs when the input exceeds the root context window or requires multi-step programmatic exploration—the neural backbone must generate additional tokens beyond the final user-visible output. These include the REPL code instructions (typically 50–200 tokens per retrieval action), the llm_batch sub-instance calls (each processing a sliced chunk of the externalized context through a separate forward pass), and the intermediate reasoning tokens between REPL operations. Empirical measurements from RLM benchmarks indicate that a typical complex query against a 1M-token document generates approximately 3,000–8,000 internal tokens to produce a 500-token final answer. This yields a recursive token multiplier of approximately 6–16×: for every user-visible output token, the backbone generates 6–16 internal tokens. On simpler queries where the input fits

within the root context window, the RLM does not activate and the multiplier is 1×—identical to a standard model.

The third cost term is the Dense Verification pass. Each refinement iteration triggers one full-capacity forward pass over the drafted output and the accumulated REPL state trace. Because the verification input is typically 1,000–3,000 tokens (the draft plus a condensed execution log), each verification pass costs approximately 16 GFLOPs × 2,000 tokens = 32 MFLOPs per verification cycle. With the recommended 2–3 refinement iterations before the convergence-based hard stop, the total verification overhead per query is approximately 64–96 MFLOPs—a fixed per-query cost that does not scale with the externalized context size.

The fourth cost term is the overhead from failed verification passes. When the Dense Verification Layer rejects a draft (due to hallucinated REPL variables, factual grounding failures, or structural incoherence), the backbone must regenerate the output, incurring an additional generation pass plus a new verification cycle. In a well-tuned production system, the rejection rate should stabilize below 15–20%, adding approximately 0.15–0.20× to the average cost multiplier.

Table 8: Computational Cost Per User-Visible Token — Full Decomposition

Cost Component	Dense Transformer 70B	Dense MoE 670B (37B active)	Mamba3-MoE-RLM 60B (8B active)	Notes for Mamba3-MoE-RLM
Base FLOPs / generated token	~140 GFLOPs	~74 GFLOPs	~16 GFLOPs	8.75× cheaper than 70B dense; Mamba-3 + 2 active experts + 8 attn layers
Recursive token multiplier (complex query)	1.0× (none)	1.0× (none)	6–16×	REPL code generation + llm_batch sub-calls + intermediate reasoning; 1.0× on simple queries
Dense Verification overhead / query	N/A	N/A	64–96 MFLOPs (2–3 iterations)	Fixed per-query cost; does not scale with external context size
Rejection/regeneration overhead	N/A	N/A	+15–20% average	When verification rejects draft; tunable via training quality and REPL code accuracy
Effective FLOPs / visible token (simple query, no RLM)	~140 GFLOPs	~74 GFLOPs	~19 GFLOPs	7.4× cheaper than 70B dense; base + 1 verification pass
Effective FLOPs / visible token (complex 1M-token query, RLM active)	~140 GFLOPs (but cannot handle 1M)	~74 GFLOPs (KV cache may OOM)	~160–256 GFLOPs	Higher than dense per-token BUT the dense model cannot serve the query at all; the hybrid can

This decomposition reveals the essential cost trade-off of the Mamba3-MoE-RLM-Dense architecture. On simple, short-context queries where the RLM does not activate, the system is approximately 7× cheaper per token than a dense 70B Transformer. On complex, long-context queries where the RLM engages recursive exploration, the per-visible-token cost rises to 1.1–1.8× the dense baseline—but this comparison is misleading because the dense baseline cannot serve the query at all (its KV cache would overflow at 1M tokens). The Mamba3-MoE-RLM-Dense architecture's higher per-token cost on complex queries is the price of capability: it is paying for 10M+ token context reach that no dense Transformer can provide at any cost.

9.4 KV Cache Scaling: The Structural Advantage

The KV cache represents the most decisive hardware differentiation between the Mamba3-MoE-RLM-Dense architecture and dense Transformers. In a standard 80-layer dense Transformer with hidden dimension 8,192, each token in the context adds approximately 2.6 MB to the cache in FP16 precision. At 128K tokens, this accumulates to over 42 GB—often exceeding the model weights as the dominant memory consumer. At 512K tokens, the cache balloons to approximately 168 GB, exceeding any single GPU's HBM capacity. At 1M tokens, it reaches 336 GB, and by 10M tokens (the RLM frontier), a hypothetical dense Transformer KV cache would require 3.3 terabytes of HBM—physically impossible on any existing or announced GPU hardware.

The Mamba3-MoE-RLM-Dense architecture eliminates this scaling wall through a three-layer defense. The first layer is the Mamba-3 MIMO backbone, which replaces 88 of the 96 total layers (at the research scale) with SSM recurrences that maintain a constant 118 MB state regardless of sequence length. The second layer is the 8 attention anchor layers, which do generate KV cache, but their cache is 91.7% smaller than a fully attentive architecture (8 layers vs. 96) and further compressed by grouped-query attention (8 KV heads instead of 32). Even at 128K tokens, this residual cache consumes only 2.1 GB. The third layer is the RLM's context externalization: when the user's input exceeds even the hybrid's comfortable attention range, the RLM moves the text entirely out of the neural network's context window and into CPU-resident REPL memory. The backbone's root context window remains at a fixed 8K–16K tokens (metadata only), generating a KV cache of approximately 130–260 MB regardless of how large the externalized prompt becomes. The user's 10M-token document consumes ~20 GB of CPU RAM—cheap, plentiful, and completely decoupled from GPU HBM.

Table 9: KV Cache Memory at Increasing Context Lengths — Dense Transformer vs. Mamba3-MoE-RLM-Dense

Context Length	Dense 80-Layer KV Cache	Mamba3-MoE-RLM Attention Cache (8 layers + GQA)	Mamba-3 Recurrent State (56 layers)	RLM REPL Memory (CPU)	Mamba3-MoE-RLM Total (GPU + CPU)
4K tokens	1.3 GB (GPU)	67 MB (GPU)	118 MB (GPU, constant)	0 (no RLM needed)	185 MB GPU total
32K tokens	10.5 GB (GPU)	0.54 GB (GPU)	118 MB (GPU, constant)	0 (no RLM needed)	0.66 GB GPU total
128K tokens	42 GB (GPU)	2.1 GB (GPU)	118 MB (GPU, constant)	0 (fits in native context)	2.2 GB GPU total
512K tokens	168 GB (infeasible on 1 GPU)	~130 MB (root window only; RLM activates)	118 MB (GPU, constant)	~1 GB (CPU)	248 MB GPU + 1 GB CPU
1M tokens	336 GB (infeasible)	~130 MB (root window only)	118 MB (GPU, constant)	~2 GB (CPU)	248 MB GPU + 2 GB CPU
10M tokens	3.3 TB (physically impossible)	~130 MB (root window only)	118 MB (GPU, constant)	~20 GB (CPU)	248 MB GPU + 20 GB CPU

At 10M tokens, the Mamba3-MoE-RLM-Dense architecture consumes 248 MB of GPU memory for its attention cache and recurrent state—a figure that is 13,300× smaller than what a dense Transformer would hypothetically require. The cost is transferred to cheap CPU RAM ($3–5/GB) rather than expensive GPU HBM ($50–100/GB-equivalent), yielding a memory cost reduction of approximately 150–300× at the 10M-token scale. This is not an incremental optimization; it is a qualitative capability threshold: the Mamba3-MoE-RLM-Dense architecture can serve queries that are physically impossible for any dense Transformer on any existing or announced hardware.

9.5 Energy, Power, and Dollar Cost Per Query

GPU power consumption during inference is proportional to both active parameter count and arithmetic utilization. An NVIDIA H100 SXM5 GPU has a thermal design power of 700W and achieves peak performance of approximately 990 TFLOPS in FP16. Dense Transformer decode dramatically under-utilizes this budget because memory-bound operations consume power without proportional compute output, achieving approximately 15–25% of peak utilization. The Mamba3-MoE-RLM-Dense architecture improves utilization through two mechanisms: LatentMoE sparsity reduces weight-loading bandwidth by the compression ratio (8×), and Mamba-3's MIMO formulation raises arithmetic intensity from approximately 2.5 ops/byte to 10–15 ops/byte, approaching the H100's compute-bound regime of 16.7 ops/byte. The combined effect yields

55–70% GPU utilization during decode, compared to 15–25% for dense models—a 2.5–4× improvement in tokens per watt-hour.

However, the RLM's recursive token multiplier partially offsets this efficiency gain on complex queries. The REPL sandbox also consumes CPU power (typically 100–200W per CPU socket), adding approximately 15–30% to the total system power draw compared to a GPU-only serving stack. When the full power envelope is accounted for—GPU, CPU, NVLink/PCIe interconnect, and DRAM refresh—the Mamba3-MoE-RLM-Dense architecture delivers approximately 2–3× better energy efficiency per simple query and approximately 0.8–1.2× energy efficiency per complex RLM-active query compared to a dense 70B Transformer. The critical nuance is that the complex query comparison is only meaningful because the hybrid can actually serve the query; the dense model cannot.

Table 10: Estimated Dollar Cost Per Query — Dense Transformer vs. Mamba3-MoE-RLM-Dense (H100 Cloud at $2.50/GPU-hour)

Query Type	Dense 70B (4× H100)	Dense MoE 670B (8× H100)	Mamba3-MoE-RLM 60B (1× H100 + CPU)	Hybrid Cost Ratio vs. Dense 70B
Simple (500 tok out, 4K context, no RLM)	$0.0028	$0.0040	$0.0004	7× cheaper
Medium (500 tok out, 128K context, no RLM)	$0.014	$0.008	$0.0009	15× cheaper (smaller KV cache = higher batch)
Complex (500 tok out, 1M context, RLM active)	Infeasible (KV OOM)	$0.042 (if cache fits)	$0.006 (RLM + 3 verifications)	7× cheaper vs MoE; ∞ vs dense (cannot serve)
Extreme (500 tok out, 10M context, RLM + parallel sub-instances)	Impossible	Impossible	$0.05–$0.12 (16 sub-instances, 3 verifications)	Only architecture capable; no comparison possible

9.6 Training Resource Requirements for the Mamba3-MoE-RLM-Dense System

Training the Mamba3-MoE-RLM-Dense architecture is a multi-stage process that differs structurally from training a monolithic dense Transformer. The total training pipeline decomposes into four phases, each with distinct hardware requirements. Phase 1 is pre-training the Mamba-3 + Attention + LatentMoE neural backbone using Phased Specialization (Section 7.1), which requires standard GPU cluster training but with differential learning rate scheduling. The research-scale 60B model requires approximately 3–5 trillion training tokens on a cluster of 256–512 H100 GPUs over 3–4 weeks, at an estimated cost of $0.5–1M. The production-scale 550B model requires 8–12 trillion tokens on 2,048–4,096 GPUs over 6–8 weeks, at approximately $6–10M. The MoE layers' sparse

activation means that per-token gradient computation is reduced by the sparsity factor (approximately 6–12× compared to a dense model of equal total parameter count), and the SSM-MoE hybrid reaches equivalent performance in approximately 2.35× fewer training steps than a pure Mamba baseline.

Phase 2 is RLM capability training via Reinforcement Learning with Dense Utility Rewards (LongR). This phase trains the backbone to generate executable REPL code, invoke llm_batch sub-instances, and interpret execution results. LongR requires a persistent REPL sandbox running alongside each training GPU, adding CPU and memory overhead to the training cluster. Context is progressively scaled from 16K to 32K tokens during curriculum learning, with each training step requiring both a generation rollout (the backbone produces a REPL trajectory) and a reward computation (an external verification model scores each intermediate retrieval action via the Relative Information Gain metric). This phase typically runs for 5,000–15,000 RL steps on 64–256 GPUs over 1–2 weeks, at an estimated cost of $100K–$300K. The dense utility reward formulation is critical: absolute density rewards cause policy hacking (verbose code to inflate prior uncertainty), while threshold density rewards lack granularity. Only the relative, constrained utility reward produces stable training convergence.

Phase 3 is Dense Verification Layer calibration. The verification layer reuses the backbone's parameters (it is not a separate model) but requires fine-tuning on a curated dataset of correct-vs-incorrect REPL trajectories to calibrate its rejection thresholds. This supervised fine-tuning phase is comparatively lightweight: 2,000–5,000 examples, 1–3 days on 8–32 GPUs, at a cost of $5K–$20K. Phase 4 is optional cross-architecture distillation (Llamba-style), which can bootstrap the entire system from a pre-trained dense Transformer teacher using less than 0.1% of the original training data, at a cost of $4K–$8K for a 3–8B student model—an order-of-magnitude reduction that dramatically lowers the barrier to entry.

Table 11: Training Pipeline Cost Decomposition — Mamba3-MoE-RLM-Dense vs. Dense Transformer

Training Phase	Architecture	GPU Type × Count	Duration	Training Tokens	CPU RAM / Node	Est. Cost	Dense 70B Equivalent Cost
Phase 1: Backbone Pre-training	Mamba3-MoE-RLM 60B	H100 × 512	3–4 weeks	3–5T	32 GB (standard)	$0.5–1M	$3–4M (70B dense)
Phase 2: RLM RL Training (LongR)	Mamba3-MoE-RLM 60B	H100 × 128	1–2 weeks	N/A (RL steps)	64–128 GB (REPL sandboxes)	$100–300K	N/A (no equivalent)
Phase 3: Verification Calibration	Same backbone	H100 × 16	1–3 days	2–5K examples	32 GB (standard)	$5–20K	N/A (no equivalent)
Phase 4: Distillation (optional)	3–8B student	H100 × 8	2–5 days	<0.1% of teacher	32 GB (standard)	$4–8K	$500K+ (from scratch)
TOTAL (Phases 1–3)	60B hybrid	—	5–7 weeks	—	—	$0.6–1.3M	$3–4M (dense 70B only)

The Mamba3-MoE-RLM-Dense training pipeline costs approximately 25–35% of training a comparable dense 70B Transformer, while producing a system with qualitatively superior capabilities (unbounded context, recursive self-correction, sparse inference). The additional phases (LongR RL training, verification calibration) add approximately $100–320K to the pipeline—costs that have no dense Transformer equivalent because these capabilities simply do not exist in the dense paradigm. The elevated CPU memory requirement during Phase 2 (64–128 GB per node for REPL sandboxes) is the primary non-GPU infrastructure cost unique to this architecture.

9.7 Scaling Laws: The Effective Parameter Ratio

The Chinchilla scaling laws established that dense Transformer performance scales as a power law with both parameter count and training data. However, these laws assume all parameters are active for every token—an assumption the Mamba3-MoE-RLM-Dense architecture violates at every level. The LatentMoE activates only 2 of 16–64 experts per token. The Mamba-3 layers process tokens via $O(N \times d)$ recurrence rather than $O(d^2)$ attention. The RLM executes focused algorithmic search rather than diffuse probabilistic attention. The cumulative effect is that the Mamba3-MoE-RLM-Dense system operates on a fundamentally steeper scaling curve than dense models.

We define the Effective Parameter Ratio (EPR) as the quality-matched dense Transformer parameter count divided by the hybrid's active parameter count. Empirical evidence from the production systems that approximate the full Mamba3-MoE-RLM design suggests that EPR

increases super-linearly with scale: at 8B active parameters, the hybrid matches dense models with approximately 25–35B parameters (EPR ≈ 3–4×). At 55B active parameters, the hybrid competes with dense models at 200–400B+ parameters (EPR ≈ 4–7×). This super-linear scaling arises because the synergy between SSM temporal processing, MoE knowledge routing, and RLM algorithmic search becomes more valuable as each component's absolute capacity grows—the MoE's larger expert pool provides richer knowledge for the RLM's search to exploit, while the Mamba-3 backbone's larger state dimension captures finer-grained temporal dynamics for the experts to specialize over.

For hardware planning purposes, the EPR means that a deployment targeting "70B-class quality" should provision for approximately 12–20B active parameters in the hybrid design (total parameters: 60–120B including dormant MoE experts), requiring 1–2 H100 GPUs plus 64–128 GB CPU RAM for the REPL—compared to 4–8 H100 GPUs for a dense 70B model with no REPL capability and no recursive self-correction. Looking forward, next-generation accelerators (NVIDIA Blackwell B200 with 192 GB HBM3e; GB200 NVL72 with 13.5 TB aggregate GPU memory) are architected around sparse, high-arithmetic-intensity workloads that precisely match the Mamba3-MoE-RLM-Dense system's computational profile, suggesting that the EPR advantage will increase, not diminish, on future silicon.

10. Production-Scale Integrations

The theoretical advantages of this multi-paradigm approach have been rapidly validated through massive, production-scale deployments by industry leaders. The consistent finding across these hybrid deployments is that a highly specific architectural ratio—typically an 8-to-15% ratio of attention layers to SSM layers—captures the optimal balance.

10.1 AI21 Jamba

AI21's Jamba model operates as a primary blueprint, functioning as a "triple hybrid" combining Transformer attention, Mamba, and MoE. It utilizes a 1:7 attention-to-Mamba ratio with MoE routing every other layer (16 experts, top-2). This allows a 52-billion parameter model to operate with only 12 billion active parameters per token, enabling a 256K-token context window with an 8× smaller KV cache, fitting on a single 80GB GPU with 3× higher throughput.

10.2 NVIDIA Nemotron-H

NVIDIA's Nemotron-H family scales the paradigm further, optimizing for real-time agentic execution. The 56B parameter model replaces the vast majority of self-attention layers with Mamba-2 blocks, ensuring constant computation and memory per generated token. It introduces FP8 training recipes and MiniPuzzle compression (pruning and distillation from 56B to 47B parameters), maintaining accuracy while improving inference speed by 20%. When augmented with LatentMoE routing and NVFP4 (4-bit) pretraining optimizations, memory requirements are drastically cut, yielding a 4×

inference speedup on Blackwell architectures. Nemotron-H-56B outperforms much larger dense models like Qwen-2.5-72B and Llama-3.1-70B by nearly 10 points on 8-shot CoT GSM8K evaluations.

10.3 Tencent Hunyuan-TurboS

Tencent's Hunyuan-TurboS represents the most exhaustive integration, scaling to 560 billion total parameters with 56 billion activated per forward pass. The architecture interweaves 128 layers, deploying 57 Mamba-2 blocks, 7 attention layers, and 64 MoE layers containing 32 specialized experts and one shared expert. It incorporates an adaptive long-short chain-of-thought mechanism trained via multi-stage RL, dynamically switching between rapid response generation and deep recursive thinking modes. Hunyuan-TurboS ranks in the top tier of Chatbot Arena leaderboards, achieving 45% lower operational cost per token than dense Transformer baselines.

10.4 Additional Hybrid Systems

NVIDIA's Nemotron-3 Nano variant (30B total / 3B active) provides 3.3× higher throughput than Qwen3-30B-A3B. Microsoft's Phi-4-mini-flash-reasoning introduces the SambaY architecture at 3.8B parameters, achieving up to 10× throughput over its Transformer counterpart and 52.29% on AIME 2024. IBM's Granite 4.0 uses a 9:1 Mamba-to-attention ratio with fine-grained MoE for enterprise-scale deployment.

Table 4: Production-Scale Hybrid Architecture Integrations

Model	Total Params	Active Params	SSM Variant	MoE Configuration	Context / KV Cache
Jamba	52B	12B	Mamba-1	1:7 Attn-to-Mamba; 16 exp, top-2	256K, 8x smaller cache
Nemotron-H	56B (47B compressed)	~12B	Mamba-2/3	Majority Mamba; MiniPuzzle pruned	Constant memory per token
Hunyuan-TurboS	560B	56B	Mamba-2	57 Mamba, 7 Attn, 64 MoE; 32+1 exp	Adaptive Chain-of-Thought
Phi-4-mini-flash	3.8B	3.8B	Mamba-1 SambaY	Synthetic RL data	10x throughput over Transformer
Nemotron-3 Nano	30B	3B	Mamba-2	LatentMoE + RL reasoning	3.3x throughput vs Qwen3-30B

11. Evaluation Paradigms for Recursive Agents

Standard evaluation benchmarks for language models—such as the MMLU suite—rely heavily on single-turn, static, multiple-choice paradigms. These benchmarks are fundamentally inadequate for

evaluating autonomous, recursive architectures because they fail to assess the model's ability to execute multi-step logic, recover from errors, or navigate programmatic environments over extended temporal horizons. Evaluating an RLM requires a shift from measuring the final output to auditing the entire cognitive trajectory. For formal logic and state tracking, ARC-AGI and MATH-500 assess the complex-valued Mamba-3 state tracking and MoE router combinatorial logic. Beyond static accuracy, specialized trajectory metrics audit the RLM's autonomous behavior: loop convergence rates, recursive stack depth stability, and exact-match success rates of intermediate REPL code executions.

Table 5: Evaluation Criteria for Recursive Agent Architectures

Evaluation Metric	Target Analysis	Vulnerability Detected
Contextual Aggregation (OOLONG)	Synthesize data across 10M+ tokens	Context rot, attention saturation
Trajectory Convergence Rate	Efficiency of the RLM path to final state	Infinite looping, redundant REPL calls
Intermediate Exact-Match	Accuracy of sandbox-generated code	Syntax errors, hallucinated variables
Dense Verification Pass Rate	Frequency of output rejection by supervisor	Logical divergence, prompt drift

12. Maintenance, Observability, and Drift Mitigation

Deploying an autonomous, recursive AI system into a production environment introduces novel failure modes absent in traditional, stateless query-response models. An agent that performs flawlessly in closed-domain testing can rapidly degrade in production—a phenomenon known as agent drift. Agent drift manifests through several distinct vectors: Concept drift occurs when user intents evolve organically; Data drift emerges when underlying APIs or databases update, breaking previously reliable RLM code generation; and Model drift involves subtle shifts in reasoning capacity following dependency updates. Because the RLM generates code and iterates autonomously, a single instance of data drift can trigger a catastrophic cascading failure, initiating a twenty-iteration retry loop that drains compute resources without producing a coherent output.

To ensure robust maintenance, the architecture demands session-level observability and highly granular telemetry. Maintainers must continuously monitor behavioral proxies: step count anomalies (if the rolling average of recursive stack depth suddenly spikes from a baseline of four steps to fifteen, the system alerts engineers to a probable infinite loop), response length volatility, and MoE routing distributions (subtle shifts can cause expert collapse where the router dispatches the vast majority of tokens to a single expert). In a production setting, the Dense Verification Layer transcends its role as a simple generation filter, operating as the primary runtime security and stability monitor. By intercepting and analyzing the intermediate states of the RLM, the verification

layer enforces strict logic across the entire trajectory, providing the most accurate real-time indicator of overall system health.

13. Challenges and Open Problems

Despite the remarkable progress documented in this paper, several critical challenges remain before a fully unified architecture is realized. Training stability is a persistent concern: MoE routing introduces discrete, non-differentiable decisions that create sparse gradients, while SSM layers require careful initialization that can interact poorly with standard Transformer training recipes. Routing collapse remains MoE's Achilles heel, and a recent 2025 finding reveals that approximately 10% of routers select different experts between training and inference frameworks.

Hardware optimization for SSMs remains nascent compared to the decades of hardware-software co-design that Transformers benefit from (FlashAttention, tensor cores, highly tuned CUDA kernels). SSMs require fundamentally different operations—parallel scans, complex-valued arithmetic, trapezoidal discretization—that have received far less optimization effort. Mamba-3's inference-first philosophy explicitly targets this gap, but GPU architectures remain optimized for dense matrix multiplication.

Recursive loop termination criteria are largely heuristic: fixed iteration counts, quality-score thresholds, or the model declaring convergence. The theoretical framework of Yang et al. (2025) provides initial formalization but does not account for reward hacking—where the generator learns to exploit the evaluator's vulnerabilities, causing score inflation without quality improvement. Finally, evaluation methodology remains inadequate; standard benchmarks (MMLU, GSM8K) are saturated by frontier models, and no existing benchmark suite adequately evaluates the compound benefits of hybrid architectures across long-context reasoning, efficiency-quality tradeoffs, recursive improvement trajectories, and expert specialization patterns simultaneously.

14. Conclusion

The architectural synthesis of Mamba-3, Latent Mixture of Experts, Recursive Language Models, and Dense Verification represents a necessary and definitive paradigm shift in artificial intelligence. As the physical and economic limitations of the quadratic dense Transformer have become insurmountable for continuous agentic operations, this multi-layered blueprint systematically dismantles those algorithmic barriers.

Three key insights emerge from this comprehensive synthesis. First, the components are complementary, not competing: SSMs handle efficient sequence processing, MoE provides scalable capacity, RL-based reasoning enables variable-depth thinking, and recursive refinement catches errors. Second, reasoning is a training methodology, not an architecture: it transfers across SSM hybrids, dense models, and MoE systems, but efficient architectures make it economically viable at scale. Third, full integration is achievable but not yet optimized: Hunyuan-TurboS demonstrates that

all four paradigms can coexist in a single production system at 560B parameters, but principled methods for joint optimization of SSM parameters, MoE routing, reasoning RL rewards, and refinement loop dynamics remain the critical frontier.

The implementation of the Mamba-3 MIMO framework, enhanced by exponential-trapezoidal discretization and complex-valued state transitions, provides the mathematical rigor and immense inference throughput necessary for sustained temporal processing without the linear memory bloat of the traditional KV cache. The integration of LatentMoE mechanisms delivers massive parameter capacity utilizing geometric down-projection to bypass catastrophic communication bottlenecks. By externalizing the context window into programmatic REPL environments, the RLM paradigm eradicates context rot entirely, transforming inefficient probabilistic attention into deterministic, algorithmically sound search protocols capable of processing millions of tokens flawlessly. The strict operational oversight of the Dense Verification Layer, combined with the dense utility rewards provided by LongR, ensures that the resulting cognitive trajectory remains grounded, coherent, and structurally sound. When optimized via Phased Specialization and deployed utilizing Disaggregated Serving on rack-scale hardware, this hybrid framework establishes an economically viable, high-throughput, and rigorously verifiable foundation capable of powering the unbound, autonomous agents defining the post-Transformer era of computational intelligence.

References

[1] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762.

[2] Gu, A., Goel, K., and Ré, C. (2022). Efficiently Modeling Long Sequences with Structured State Spaces. ICLR 2022. arXiv:2111.00396.

[3] Gu, A. and Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. COLM. arXiv:2312.00752.

[4] Dao, T. and Gu, A. (2024). Transformers are SSMs: Generalized Models Through Structured State Space Duality. ICML 2024. arXiv:2405.21060.

[5] Lahoti, P. et al. (2026). Mamba-3: Improved Sequence Modeling using State Space Principles. ICLR 2026. arXiv:2603.15569.

[6] Jiang, A. et al. (2024). Mixtral of Experts. Mistral AI. arXiv:2401.04088.

[7] DeepSeek-AI. (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437.

[8] DeepSeek-AI. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. Nature 2025. arXiv:2501.12948.

[9] Lieber, O. et al. (2024). Jamba: A Hybrid Transformer-Mamba Language Model. AI21 Labs. ICLR 2025. arXiv:2403.19887.

[10] NVIDIA. (2025). Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models. arXiv:2504.03624.

[11] Tencent Hunyuan Team. (2025). Hunyuan-TurboS: Advancing LLMs through Mamba-Transformer Synergy and Adaptive Chain-of-Thought. arXiv:2505.15431.

[12] Microsoft. (2025). Phi-4-mini-flash-reasoning. arXiv:2507.06607.

[13] Recursive Language Models. arXiv:2512.24601.

[14] Prime Intellect. (2026). Recursive Language Models: the paradigm of 2026. primeintellect.ai/blog/rlm.

[15] Mamba-3 Part 1 | Tri Dao. tridao.me/blog/2026/mamba3-part1/.

[16] Mamba-3 Part 2 - Methodological Deep Dive. Goomba Lab. goombalab.github.io/blog/2026/mamba3-part2/.

[17] Phased Specialization: Unlocking Hybrid Sequence Models via Optimization-Aware Training. medium.com/@mbonsign.

[18] LongR: Unleashing Long-Context Reasoning via Reinforcement Learning with Dense Utility Rewards. arXiv:2602.05758.

[19] Llamba: Scaling Distilled Recurrent Models for Efficient Language Processing. arXiv:2502.14458.

[20] Oolong: Evaluating Long Context Reasoning and Aggregation Capabilities. OpenReview.

[21] Think, But Don't Overthink: Reproducing Recursive Language Models. arXiv:2603.02615.

[22] NVIDIA Nemotron 3 Super: Complete Guide To Hybrid MoE Agentic AI. marketingagent.blog (2026).

[23] Snell, C. et al. (2024). Scaling LLM Test-Time Compute Optimally. arXiv:2408.03314.

[24] Lightman, H. et al. (2023). Let's Verify Step by Step. ICLR 2024. arXiv:2305.20050.

[25] Madaan, A. et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023. arXiv:2303.17651.

[26] Huang, J. et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR 2024.

[27] Yang, Z. et al. (2025). Can LLMs Correct Themselves? EMNLP 2025. arXiv:2510.16062.

[28] Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. arXiv:2201.11903.

[29] Zelikman, E. et al. (2022). STaR: Bootstrapping Reasoning With Reasoning. NeurIPS 2022. arXiv:2203.14465.

[30] Bai, Y. et al. (2022). Constitutional AI: Harmlessness from AI Feedback. Anthropic. arXiv:2212.08073.

[31] NVIDIA. Introducing Nemotron 3 Super. developer.nvidia.com/blog.

[32] IBM. (2025). Granite 4.0: Hyper-efficient, High Performance Hybrid Models for Enterprise.

[33] Scaling State-Space Models on Multiple GPUs with Tensor Parallelism. arXiv:2602.21144.

[34] QuadSentinel: Sequent Safety for Machine-Checkable Control in Multi-agent Systems. OpenReview.

[35] Recursive Language Models: MIT's Solution to Infinite Context. medium.com/@reliabledataengineering (2026).